

Algorytmy w Inżynierii Danych

Programie, napisz się sam¹!

Bartosz Chaber

¹...tylko najpierw powiem ci jak ;)

Metaprogramowanie

Istotnym dziedzictwem języka Lisp widocznym w języku Julia jest możliwość reprezentacji swojego własnego kodu w postaci struktur danych. Pozwala to na **modyfikację** kodu i jego dynamiczne konstruowanie.

Poniższy kod jest przykładem makra w Racket Lisp:

```
(define-syntax-rule (swap x y) (swap first last))
```

```
(let ([tmp x])  
  (set! x y)  
  (set! y tmp)))
```

```
(let ([tmp first])  
  (set! first last)  
  (set! last tmp)))
```

Symbol i wyrażenie

Symbole poprzedzone są znakiem `:`, np. `:foo`, `:sort!` i odpowiadają nazwom identyfikatorów w Julia (nazw funkcji, nazw zmiennych).

Wyrażenia, podlegają parsowaniu, kompilacji i uruchomieniu. Istnieje jednak możliwość wstrzymania przetwarzania wyrażeń tuż po parsowaniu.

```
x = 5 # parsowanie, kompilacja, uruchomienie
:(x = 5) # parsowanie
quote x = 5 end # parsowanie
```

Użycie : (wyrażenie) lub quote wyrażenie end zwraca abstrakcyjne drzewo składni, sparsowanego wyrażenia. Istnieje funkcja eval(...), która pozwala na kompilację i uruchomienie przekazanego wyrażenia.

```
x = 5
:(x = 10)
println(x) # 5
eval(:(x = 10))
println(x) # 10
```

Wyrażenie składa się z głowy (head, wartość jest symbolem), oraz z listy argumentów (args, wartości mogą być wyrażeniami, symbolami lub stałymi wartościami).

`:(x + 5)`

Expr

```
head: Symbol call
args: Array{Any}((3,))
  1: Symbol +
  2: Symbol x
  3: Int64 5
```

`:(true ? +1 : -1)`

Expr

```
head: Symbol if
args: Array{Any}((3,))
  1: Bool true
  2: Int64 1
  3: Int64 -1
```

Makra

Operują na abstrakcyjnym drzewie składni (po parsowaniu, przed kompilacją). Makro nie ma wiadomości o typie ani wartości zmiennych.

```
julia> @Meta.dump x = sin(5.0)
```

```
Expr  
head: Symbol =  
args: Array{Any}((2,))  
  1: Symbol x  
  2: Expr  
    head: Symbol call  
    args: Array{Any}((2,))  
      1: Symbol sin  
      2: Float64 5.0
```

Higiena makr w Julii polega na tym, że domyślnie rozwijane makra **nie nadpisują** zmiennych/funkcji.

```
macro foo(expr)
  code = Expr(:block)
  push!(code.args, :(a = 0))
  push!(code.args, esc(expr))
  push!(code.args, :(b = 10))
  return code
end
```

```
julia> @macroexpand @foo b = 5
quote
  var"#1#a" = 0
  b = 5
  var"#2#b" = 10
end
```

Możemy też łatwo utworzyć nowy symbol, nie powodujący kolizji:

```
julia> s = gensym()  
Symbol("##226")  
julia> @eval $s = 10;  
julia> var"##226"  
10  
julia> gensym(:op)  
Symbol("##op#228")  
julia> Symbol(:y, 999)  
Symbol(:y999)
```


Przetwarzanie surowych wyrażeń jest znacząco ułatwione poprzez wykorzystanie biblioteki MacroTools.

Pozwala ono przechwycić fragmenty wyrażenia według wzorca:

```
using MacroTools
loop = quote
    for i = 1:9
        println(i)
    end
end
```

```
@capture(loop, for symbol_ = start_ : stop_ body__ end)
```

Po uruchomieniu powyższego kodu, w zmiennych `symbol`, `start`, `stop` oraz `body` będą znajdować się fragmenty definicji pętli `for`.

Spróbujmy napisać makro, które rozwinie krótkie pętle:

```
macro unroll(loop)
```

```
...
```

```
end
```

```
@unroll for i=3:6
```

```
    println(i)
```

```
end
```

```
println(3)
```

```
println(4)
```

```
println(5)
```

```
println(6)
```

```
using MacroTools: postwalk, @capture
macro unroll(loop)
  res = @capture(loop, for sym_ = start_ : stop_ body__ end)
  @assert(res == true, "Couldn't match a for-loop")
  code = Expr(:block)
  for it in start:stop
    for st in body
      expr = postwalk(e -> e == sym ? it : e, st)
      push!(code.args, expr)
    end
  end
  return esc(code)
end
```

Funkcje generowane

Pozwalają na ostateczne przepisanie kodu pomiędzy pierwszym wywołaniem funkcji, a jej kompilacją. **Mają dostęp do typów** (ale nie wartości) zmiennych. (Rada: używaj `Core.print` zamiast `println`)

```
@generated function f(t::Tuple)
    for p in t.parameters
        println(p) # (kompilacja)
    end
    quote
        println(t) # (uruchomienie)
    end
end
```

```
julia> f((1.0, 'δ', 5))
Float64
Char
Int64
(1.0, 'δ', 5)

julia> f((1.0, 'δ', 5))
(1.0, 'δ', 5)
```

Różniczkowalne programowanie

W jaki sposób zrealizować bibliotekę do
automatycznego różniczkowania ogólnego
przeznaczenia?

Jak przechowywać obliczenia?

Dość wygodnym sposobem jest *lista Wengerta* (**ang. Wengert list**).

Przykładowa lista dla wyrażenia $\cos(0.2 + \sin(x))$.

```
y1 = sin(x)
```

```
y2 = 0.2 + y1
```

```
y3 = cos(y2)
```

Jak przedstawić w niej instrukcje warunkowe lub pętle? Tak jak w grafie obliczeniowym, uruchamiając je w czasie działania, w postaci *taśmy obliczeń*.

```
function poteguj(x, n)
    r=1
    while n > 0
        n -= 1
        r *= x
    end
    return r
end
```

Dla $x=5$ i $n=3$:

```
y1 = 1
y2 = y1 * x
y3 = y2 * x
y4 = y3 * x
```

Jest też inny sposób, uogólnienie listy Wengerta: postać jednokrotnego, statycznego przypisania: *SSA-form*².

```
function f(a, b)
  if a > 0
    b = 10 + b
  end
  return a * b
end
```

```
@code_lowered f(1, 2)
CodeInfo(
  1 —      b@_4 = b@_3
    |      %2 = a > 0
    └      goto #3 if not %2
  2 —      b@_4 = 10 + b@_4
  3 ... %5 = a * b@_4
    └      return %5
)
```

²SSA – ang. Static Single-Assignment

Propagatory

Funkcje, propagujące gradient³:

pullback propagator w tył, oblicza VJP⁴

pushforward propagator w przód, oblicza JVP⁵.

Table 1. Pullbacks for some simple mathematical functions.

FUNCTION	PULLBACK
$y = a + b$	$(\bar{y}, \bar{y})$
$y = a \times b$	$(\bar{y} \times b, \bar{y} \times a)$
$y = \sin(x)$	$\bar{y} \times \cos(x)$
$y = \exp(x)$	$\bar{y} \times y$
$y = \log(x)$	$\bar{y}/x$

³<https://juliadiff.org/ChainRulesCore.jl/stable/math/propagators.html>

⁴**ang. vector-Jacobian product** – definiowane jako $v \cdot J^T$

⁵**ang. Jacobian-vector product** – definiowane jako $J \cdot v$

Poniższy operator J zwraca zarówno wartość operatora, jak i jego propagator w tył⁶:

```
J(::typeof(asin), x) = (asin(x), (Δ) -> tuple(Δ * inv(sqrt(1.0 - x^2))))
J(::typeof(sin), x) = (sin(x), (Δ) -> tuple(Δ * cos(x)))
J(::typeof(+), x, y) = (x + y, (Δ) -> tuple(Δ, Δ))
J(::typeof(*), x, y) = (x * y, (Δ) -> tuple(Δ * y, Δ * x))
J(::typeof(/), x, y) = (x / y, (Δ) -> tuple(Δ * 1 / y, -Δ * a / y / y))
J(::typeof(^), x, y) = (x ^ y, (Δ) -> tuple(Δ * y * x ^ (y-1), nothing))
```

Legenda: Δ oznacza gradient przychodzący od wyniku, $\bar{x}$ oznacza pochodną wyniku z względem x , tj. $\frac{dz}{dx}$.

⁶<https://juliadiff.org/ChainRulesCore.jl/v0.8/FAQ.html#Why-does-rrule-return-the-primal-function-evaluation?-1>

```
%1 = sin(%x)
```

```
%2 = 0.2 + %1
```

```
%3 = asin(%2)
```

```
# primal trace
```

```
x = 3;
```

```
a, B_a = J(sin, x);
```

```
b, B_b = J(+, 0.2, a);
```

```
c, B_c = J(asin, b)
```

```
@show c ≈ asin(0.2 + sin(x))
```

```
# adjoint trace
```

```
 $\bar{c}$ , = 1.0 #  $\partial c / \partial c$ 
```

```
 $\bar{b}$ , = B_c( $\bar{c}$ ) #  $\partial c / \partial b$ 
```

```
_,  $\bar{a}$  = B_b( $\bar{b}$ ) #  $\partial c / \partial a$ 
```

```
 $\bar{x}$ , = B_a( $\bar{a}$ ) #  $\partial c / \partial x = \partial f / \partial x$ 
```

```
@show  $\bar{x}$  ≈ -1.0531613736418153
```

```
%1 = %b * %b
```

```
%2 = %a + %1
```

```
%3 = %a / %2
```

```
# primal trace
```

```
a = 1.3
```

```
b = 2.5
```

```
y_1, B_1 = J(*, b, b)
```

```
y_2, B_2 = J(+, a, y_1)
```

```
y_3, B_3 = J(/, a, y_2)
```

```
@show y_3 ≈ a / (a + b^2)
```

```
# adjoint trace
```

```
ȳ_3 = 1.0
```

```
ā_1, ȳ_2 = B_3(ȳ_3)
```

```
ā_2, ȳ_1 = B_2(ȳ_2)
```

```
ḃ_1, ḃ_2 = B_1(ȳ_1)
```

```
ā = ā_1 + ā_2
```

```
ḃ = ḃ_1 + ḃ_1
```

```
@show ā ≈ 1 / y_2 - a / y_2^2
```

```
@show ḃ ≈ 2b * ȳ_2
```

Różniczkowanie instrukcji warunkowych

Jak pogodzić instrukcje warunkowe z jednokrotnym przypisaniem?

Odpowiedzią jest węzeł ϕ , zaimplementowany m.in. w LLVM⁷

Wracając do przykładu funkcji:

```
function f(a, b)
  if a > 0
    b = 10 + b
  end
  return a * b
end
```

⁷<https://mapping-high-level-constructs-to-llvm-ir.readthedocs.io/en/latest/control-structures/ssa-phi.html>

```
define i64 @julia_f(i64 signext %0, i64 signext %1) #0 {
top:
    %2 = icmp slt i64 0, %0
    %3 = xor i1 %2, true
    br i1 %3, label %top.edge, label %cond
top.edge:                                ; preds = %top
    br label %default
cond:                                    ; preds = %top
    %4 = add i64 10, %1
    br label %default
default:                                ; preds = %cond, %top.edge
    %value_phi = phi i64 [ %4, %cond ], [ %1, %top.edge ]
    %5 = mul i64 %0, %value_phi
    ret i64 %5
}
```

Przykład sieci konwolucyjnej:

```
net = Chain(  
    Conv((3, 3), 1 => 6, relu),  
    MaxPool((2, 2)),  
    Conv((3, 3), 6 => 16, relu),  
    MaxPool((2, 2)),  
    Flux.flatten,  
    Dense(400 => 84, relu),  
    Dense(84 => 10, identity),  
)
```

Przykład sieci rekurencyjnej:

```
net = Chain(  
    RNN((14 * 14) => 64, tanh),  
    Dense(64 => 10, identity),  
)
```

Podsumowanie ● ○ ○ ○

Oddzielając budowanie kodu od jego uruchomienia można ograniczyć wpływ nieefektywnego kodu obliczeniowego,

Podsumowanie ● ● ○ ○

Różniczkowanie oparte o kod źródłowy
pozwała na wygenerowanie przebiegu
dołączonego (*adjoint*) przed uruchomieniem
przebiegu pierwotnego (*primal*),

Podsumowanie ● ● ● ○

Im prostszy zestaw różniczkowalnych instrukcji, tym łatwiej napisać kod generujący przebieg dołączony.

Podsumowanie ● ● ● ●

Istotną rzeczą dla wydajności biblioteki jest
gdzie trzymamy wyniki obliczeń.

Literatura

- Michael J. Innes, Diff-ZOO, 2018, url: <https://github.com/MikeInnes/diff-zoo>
- Michael J. Innes, Don't unroll adjoint: differentiating SSA-form programs, 2019, url: <https://arxiv.org/pdf/1810.07951.pdf>
- Bart van Merriënboer, Alexander B Wiltschko, Dan Moldovan, Tangent: Automatic Differentiation Using Source Code Transformation in Python, 2017, url: <https://arxiv.org/pdf/1711.02712.pdf>
- Build your own AD, url: <https://dfdx.github.io/Yota.jl/dev/design/>